MVP Development Plan

From zero to an owner-testable booking platform in 30 days

Goal: a working, quality direct-booking site that a handful of owner friends can use end-to-end within one month — direct bookings, calendar sync, damage deposits, and the hooks for guest screening and damage coverage — built on a stack that scales to thousands of owners without a rewrite.

The One Decision That Makes a Month Possible

Run the friends test in Stripe TEST mode.

Your friends complete real, full bookings — pick dates, sign the agreement, "pay" with Stripe test cards, see the deposit held and released. But no real money moves, no real identity verification is required, and you take on zero fund-custody or compliance risk during the test.

This pulls the three slowest, riskiest items — KYC onboarding, real escrow custody, and legal exposure — off the critical path. You flip to live money in a deliberate pilot step after the test, once the flow is proven.

Everything below is scoped to get to that test. The full vision (public mode, property verification, guest CRM, referrals) comes after.

The Stack — and Why It's the Right Bet

These are the tools from the project's tech decisions. Each was chosen because it is **AI-friendly** (excellent docs, huge training footprint, so Claude Code can scaffold it fast) and **scalable** (the same code that serves 5 owners serves 5,000).

Layer	Tool	Why it wins for an AI-built MVP
Framework	**Next.js (App Router)**	The most AI-codegen-friendly web framework; one codebase for site + API
Hosting	**Vercel**	Push to GitHub → deployed. Zero DevOps. Cron jobs built in for calendar sync
Database + Auth + Storage	**Supabase**	Postgres + auth + file storage in one. Row-level security. Scales to millions of rows

Payments + Escrow	**Stripe Connect (Custom)**	Delayed payouts = escrow. Deposit holds. Owner KYC handled by Stripe, not us
Email	**Resend**	Clean API, AI-friendly templates, generous free tier
Calendar sync	**iCal (.ics)**	Standard. Works with Airbnb/VRBO/Booking.com with no per-platform dev
E-signature	**DocuSeal**	Open-source rental agreements; self-host free or cloud
Screening / Coverage	**Truvi (Superhog)**	US + Mexico support; integrated post-test (needs a partnership agreement)
AI development	**Claude Code**	Builds and iterates the whole stack from this plan

> **Accelerator:** don't start from a blank page. Begin from a Next.js + Supabase + Stripe SaaS starter template, then have Claude Code reshape it to the booking domain. This saves roughly a week of auth/billing boilerplate.

What's Real vs. Stubbed for the Test

Being honest about this is what keeps the timeline real and the quality high.

Capability	In the 30-day test	The honest reason
Direct booking flow	**Fully real**	This is the whole point
Calendar sync (inbound iCal)	**Fully real**	Core to avoiding double-bookings
Calendar export (outbound .ics)	**Real**	Cheap to add; one endpoint
Payments + escrow	**Real, in test mode**	Stripe test mode proves the flow without fund custody
Damage deposit	**Real, in test mode**	Stripe authorization hold or held charge
Rental agreement	**Fully real**	DocuSeal works the same in test
Automated emails	**Fully real**	Resend works the same in test
Guest screening (Truvi)	**Designed + feature-flagged; sandbox only**	Truvi needs a signed partnership + onboarding call — won't be live in 30 days. We build the integration point and turn it on later
Damage coverage (insurance)	**Stripe deposit stands in; real coverage deferred**	Third-party damage underwriting requires a commercial contract. The deposit-hold mechanic is the test stand-in
Owner identity KYC	**Skipped in test mode; built for pilot**	Not needed when no real money moves
Public mode / property verification	**Deferred**	Friends = trusted-guest mode only

Cost to Run the Test Month

With test mode + free tiers, the platform itself is **effectively free**. Your only real spend is the AI subscription that builds it, plus the domain if you don't already own it.

Item	Test-month cost	Notes
Vercel (Hobby tier)	**$0**	Free for the test; upgrade to Pro ($20/mo) at the live pilot
Supabase (Free tier)	**$0**	500 MB DB + 1 GB storage — plenty for a handful of owners

Stripe (test mode)	**$0**	No charges in test mode
Resend (Free tier)	**$0**	3,000 emails/mo
DocuSeal (self-hosted or free)	**$0**	Self-host on Supabase/Railway, or open-source instance
Monitoring (Sentry free)	**$0**	Error tracking
GitHub	**$0**	Already set up
Domain (familiarguest.com)	**~$0–$12/yr**	Likely already owned
Claude Code (the builder)	**$20–$100**	Pro at $20/mo for light use; Max 5× at $100/mo for a full month of heavy building (recommended)
Total out of pocket	**≈ $20–$112**	Essentially just the AI subscription

> **Reality check on "scalable but free":** the free tiers are real and generous, but the moment you go live with real money and real owners, plan on the baseline below. Nothing about the code changes — you're just upgrading plan tiers.

Baseline once live (post-test, real operations)

Item	Monthly	Trigger to upgrade
Vercel Pro	$20	Custom domain on commercial use, cron limits
Supabase Pro	$25	>500 MB data, daily backups, no project pausing
Resend Pro	$20	>3,000 emails/mo
DocuSeal Cloud (or self-host $0)	$0–$20	Convenience vs. maintenance
Monitoring	$0	Free tier holds for a long time
Baseline total	**≈ $65–$85/mo**	Matches the Year-1 infrastructure line in the financial model

Plus per-transaction: Stripe ~2.9% + $0.30 per booking (passed to owners), and Truvi ~$4–5 per screened booking (passed to guests as the $5 add-on) once that partnership is live.

The 30-Day Build Plan

A four-week sprint, each week ending in something demonstrable. Claude Code does the building; you steer, review, and test as the product manager.

Week 0 — Foundation (2–3 days, runs into Week 1)

- Scaffold Next.js + Supabase + Stripe from a SaaS starter template
- Wire up GitHub → Vercel auto-deploy (staging + production)
- Define the database schema: `owners, properties, listings, guests, bookings, agreements, calendar_feeds, deposits`
- Set up Supabase Auth (email magic links — no passwords)
- Apply brand design tokens (Fraunces / Hanken Grotesk, the green/terracotta/cream palette)

Deliverable: a deployed, branded skeleton with login that you can visit at a URL.

Week 1 — Owner Onboarding & Listings

- Owner signup + dashboard
- Property/listing creation form
- Photo upload (drag-and-drop + mobile; cloud OAuth deferred to post-test)
- AI-written description (owner pastes their text or answers prompts → Claude API polishes)
- Stripe Connect account creation wired in (test mode — KYC stubbed)

Deliverable: an owner can create an account and publish a listing with photos and a description.

Week 2 — Calendar Sync & Booking Page

- **Inbound iCal:** owner pastes Airbnb/VRBO export URLs; Vercel Cron refreshes every 15–30 min; availability merged
- **Outbound .ics:** unique export URL per listing
- "Last synced" timestamp shown to set expectations
- Guest-facing **booking page** — branded to the property, mobile-first, no guest account required
- Date selection against live availability + transparent price breakdown

Deliverable: a guest can open an owner's link and see real availability and pricing.

Week 3 — Payments, Deposits & Agreements

- Stripe payment flow (test mode) with **escrow** via delayed payout
- **Damage deposit** as a separate authorization hold / held charge, with release after the inspection window
- **DocuSeal** rental agreement auto-generated per booking; guest signs before payment confirms

- Booking confirmation + state machine (`pending → agreement signed → paid → confirmed`)

Deliverable: a complete booking — dates → agreement → payment → deposit hold → confirmation — works end to end.

Week 4 — Messaging, Screening Hooks & Polish

- **Resend** automated emails: confirmation, pre-arrival, check-in instructions, checkout, deposit release
- Digital house manual (private link)
- **Guest screening / damage coverage:** build the feature-flagged integration points (Truvi sandbox); show the $5 screening and $19.99 protected-booking options in the UI even if backed by sandbox during the test
- Error handling, friendly messages, mobile QA, accessibility pass
- Seed 2–3 real owner listings; recruit the friend testers

Deliverable: the full owner + guest experience, ready for friends to test.

Pre-Test Checklist

- [ ] Staging and production environments both deploy cleanly from `main`
- [ ] All secrets in Vercel env vars; nothing committed (`.env.example` documented)
- [ ] Stripe in **test mode**; test card numbers documented for testers
- [ ] At least 2 real owner listings live with photos and calendars synced
- [ ] One full booking completed by you, start to finish, before any friend touches it
- [ ] A short "how to test" guide for your owner friends (with test card numbers)
- [ ] Feedback capture (a simple form or shared doc) ready

What Comes After the Test

The test answers one question: *is this easy and trustworthy enough that owners would actually send their guests the link?* Once you have that:

1. **Live pilot** — flip Stripe to live mode, turn on real owner KYC (Connect Custom), upgrade to paid tiers, run a small number of real-money bookings with the most enthusiastic testers.
2. **Sign the Truvi partnership** — turn guest screening and damage coverage from sandbox to real (this can run in parallel during the test month).
3. **Safety-critical review** — spend the budgeted $5,000 on a contract engineer to review Stripe webhooks, escrow timing, and security hardening before scaling.
4. **Guest CRM & rebooking** — build the retention moat (re-invite, one-click rebook, referrals).

5. **Public mode** — property verification + open listings, once trusted-guest retention is proven.

> **Bottom line:** you can have owner friends testing a genuinely good product in 30 days for the cost of one AI subscription — roughly **$20–$100** — on infrastructure that scales straight through to the Year-3 numbers in the business plan, with no rewrite.
